

TP1 Utilisation d'un ORM (EF6)

Avec PostgreSQL

Créer un nouveau projet console sous Visual Studio.

Accéder à la console du gestionnaire de package

Le gestionnaire de package permet de gérer des dépendances, d'ajouter des bibliothèques pour enrichir les fonctionnalités disponibles pour le développeur.

Outils > Gestionnaire de package NuGet > Console du gestionnaire de package

Pour installer un package, utiliser la commande

`Install-Package <nomDuPackage>`

Utiliser le gestionnaire de paquets NuGet pour Installer le connecteur PostgreSQL avec NuGet.

Ajouter les dépendances suivantes :

- `Npgsql.EntityFrameworkCore.PostgreSQL`
- `Microsoft.EntityFrameworkCore.Design`
- `Microsoft.EntityFrameworkCore.Tools`

Documentation du connecteur : <https://www.npgsql.org/efcore/>

Créer une classe BloggingContext avec le code ci-dessous :

```
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;

namespace ConsoleApp.PostgreSQL
{
    public class BloggingContext : DbContext
    {
        public DbSet<Blog> Blogs { get; set; }
        public DbSet<Post> Posts { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
        =>
optionsBuilder.UseNpgsql("Host=my_host;Database=my_db;Username=my_user;Password=my
_pw");
    }

    public class Blog
    {
        public int BlogId { get; set; }
        public string Name { get; set; }
        public string Url { get; set; }

        public List<Post> Posts { get; set; }
    }

    public class Post
    {
        public int PostId { get; set; }
        public string Title { get; set; }
        public string Content { get; set; }

        public int BlogId { get; set; }
        public Blog Blog { get; set; }
    }
}
```

Cette ligne correspond à la connexion string (chaîne de connexion)

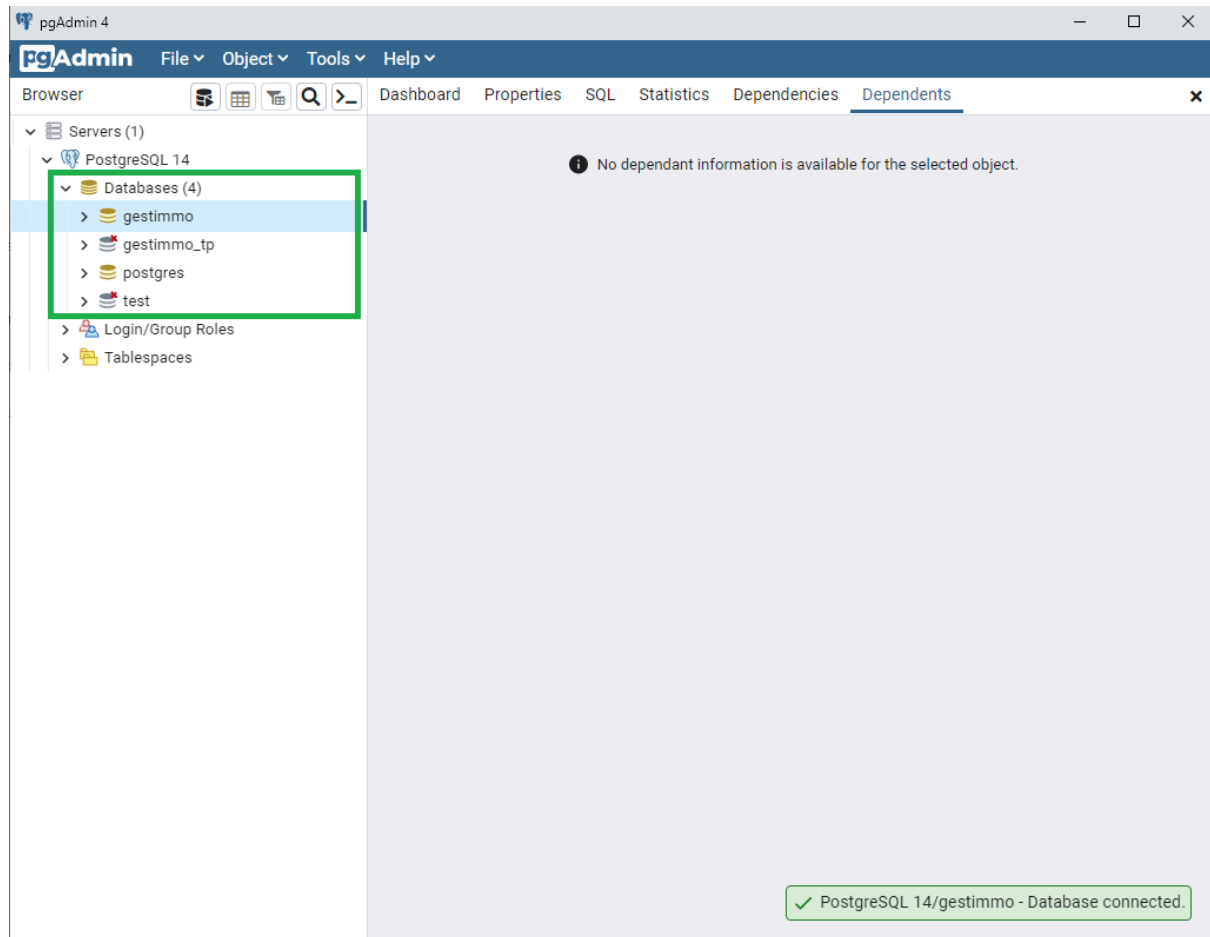
```
optionsBuilder.UseNpgsql("Host=my_host;Database=my_db;Username=my_user;Password=my
_pw");
```

Vous devez remplir les valeurs tels que :

- Host : adresse ip du serveur (généralement localhost car base en local)
- Database : le nom de la base de données
- Username : le nom de l'utilisateur (par défaut postgres)
- Password : le mot de passe (root par défaut)

Bien évidemment, il est nécessaire de créer une base de données sur PostgreSQL via pgAdmin.

Figure 1 : Consulter les bases de données avec pgAdmin



Pour en créer une nouvelle, clique droit sur "Database", puis "Create" et "Database".

Pour consulter les tables et données d'une base de données.

Cliquer sur la base de données puis "Schemas", "public" et "Tables".

Pour voir les données d'une table.

Cliquer sur la table puis sur "View/Edit Data" puis "All rows". Ou écrire une requête SQL dans l'onglet SQL

Figure 2 : Visualisation des données d'une table

The screenshot displays the pgAdmin 4 interface. On the left, the 'Browser' pane shows the database structure: 'gestimmo' > 'Schemas (1)' > 'public' > 'Tables (12)'. The 'Appartement' table is selected. The main pane shows the 'Data output' tab with a single row of data for the 'Appartement' table. The SQL query editor at the top shows the following query:

```
1 SELECT * FROM public."Appartement"
2 ORDER BY "BienId" ASC
```

The 'Data output' table has the following columns and values:

BienId [PK] integer	AppartementId integer	Etage integer	Assenseur boolean	ChauffCommun boolean
1	3	0	20	false

The status bar at the bottom indicates 'Total rows: 1 of 1' and 'Query complete 00:00:00.430'.

Dans votre Program.cs, ajouter le code suivant pour tester l'ORM :

```
using (var db = new BloggingContext())
{
    // Create and save a new Blog
    Console.WriteLine("Enter a name for a new Blog: ");
    var name = Console.ReadLine();

    var blog = new Blog { Name = name, Url = "hello" };
    db.Blogs.Add(blog);
    db.SaveChanges();

    // Display all Blogs from the database
    var query = from b in db.Blogs
                orderby b.Name
                select b;

    Console.WriteLine("All blogs in the database:");
    foreach (var item in query)
    {
        Console.WriteLine(item.Name);
    }

    Console.WriteLine("Press any key to exit...");
    Console.ReadKey();
}
```

Avant d'exécuter votre programme, effectuer une première migration.

Créer une nouvelle migration avec la commande : `Add-Migration InitialCreate`
 InitialCreate est le nom de notre migration. Vous pouvez constater qu'un dossier Migration a été créé avec un fichier de code correspondant.

Pour lancer la migration, exécuter : `Update-Database`

Source : <https://learn.microsoft.com/fr-fr/ef/core/managing-schemas/migrations/?tabs=vs>

Vous pouvez maintenant lancer votre programme.

Saisissez un nom de blog et appuyez sur entrée, vérifiez la nouvelle ligne dans la base de données.

Ci-dessous un **résumé du fonctionnement d'EF6** :

1 Développement	2 <code>Add-Migration</code>	3 <code>Update-Database</code>
Faire des modifications de modèles	Créer une migration	Mettre à jour la base de données